50 NumPy Fresher-Level Scenario-Based Interview Questions

Resnur AI

1 Import NumPy

Scenario: Your interviewer asks you to start working with NumPy.

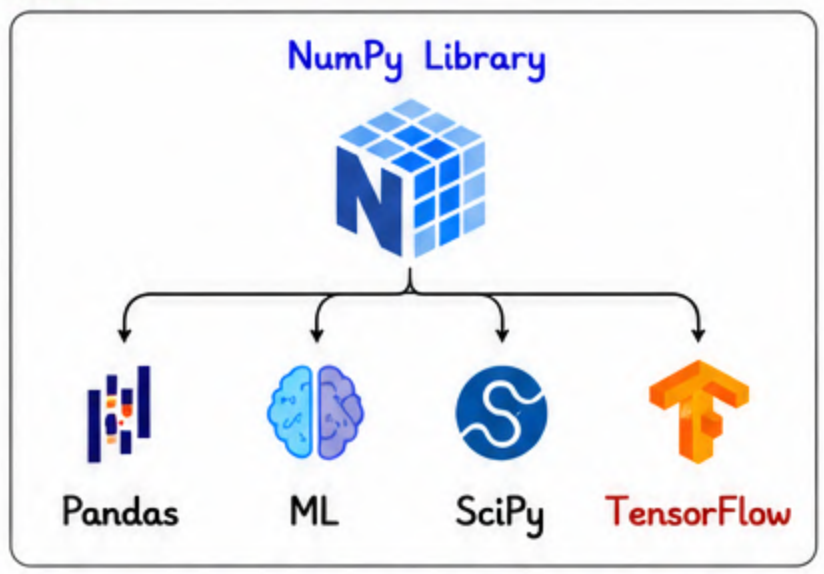
Question: How do you import NumPy in Python?

Answer:
```
import numpy as np
```

Explanation:
→ NumPy stands for Numerical Python.
→ It is the foundation of Pandas, Machine Learning, OpenCV, TensorFlow and many scientific libraries.

Why it's important: Importing NumPy is the first step to perform fast and efficient numerical computations.

Visual:



2 Create a NumPy Array

Scenario: You receive sales values and want to perform calculations efficiently.

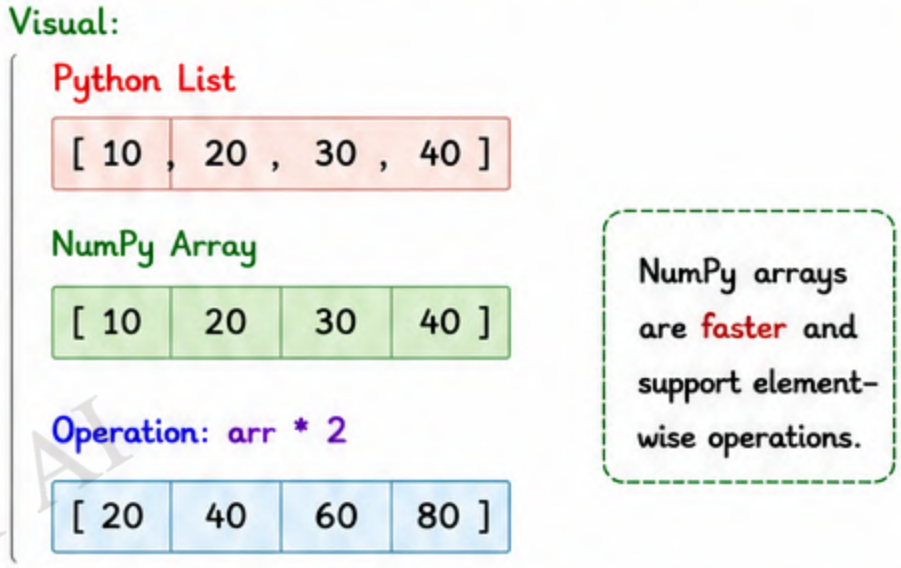
Question: How do you create a NumPy array?

Answer:
```
arr = np.array([10, 20, 30, 40])
```

Explanation:
→ NumPy arrays are faster than Python lists.
→ Supports mathematical operations directly.

Why it's important: Arrays are the core data structure in NumPy and the basis for all numerical operations.

Visual:



NumPy arrays are faster and support element-wise operations.

3 Find Shape of Array

Scenario: You are working with image data represented as a 2D array.

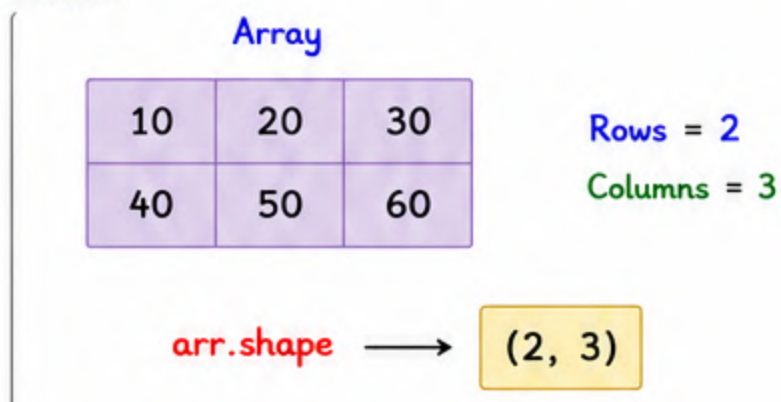
Question: How do you find the dimensions (rows, columns) of a NumPy array?

Answer:
```
arr.shape
```

Explanation:
→ shape returns a tuple (rows, columns).
→ Helps in understanding the structure of data.

Why it's important: Crucial for data preprocessing, reshaping and handling real-world datasets.

Visual:



4 Check Number of Dimensions

Scenario: You have arrays of different dimensionality and want to check how many dimensions they have.

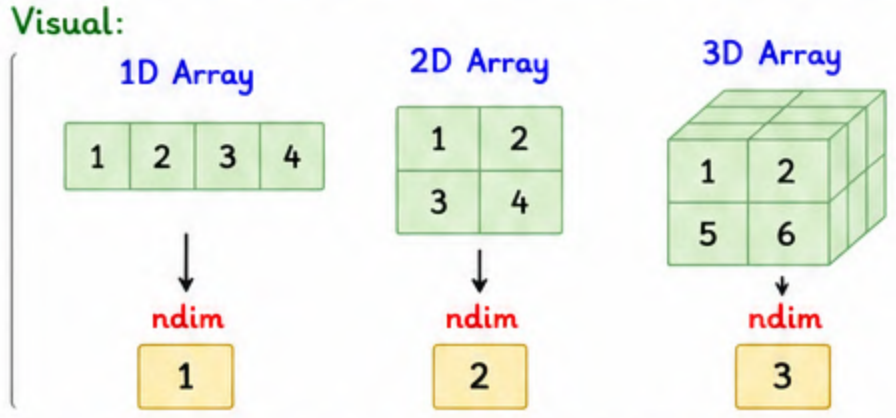
Question: How do you check the number of dimensions?

Answer:
```
arr.ndim
```

Explanation:
→ ndim returns the number of axes (dimensions).
→ 1D, 2D, 3D ... arrays are identified easily.

Why it's important: Helps in writing generic code that works for arrays of any dimension.

Visual:



5 Find Size (Total Elements) of Array

Scenario: You need to know how many total elements are present in the array.

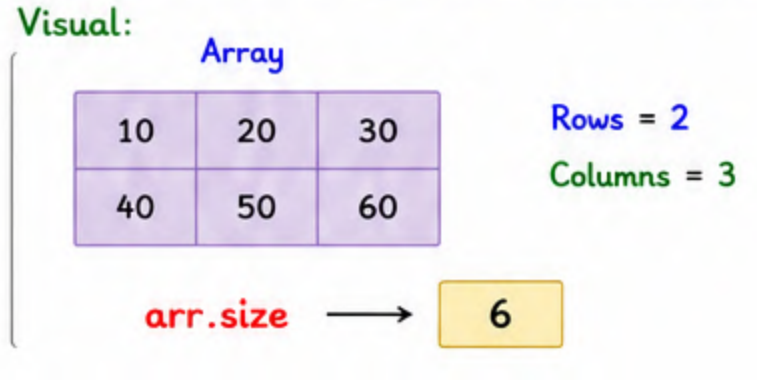
Question: How do you find the total number of elements?

Answer:
```
arr.size
```

Explanation:
→ size returns the total number of elements.
→ Useful for memory estimation and iteration.

Why it's important: Important for performance tuning and handling large datasets.

Visual:



50 NumPy Fresher-Level Scenario-Based Interview Questions

Resnur AI

6 Create an Array of Zeros

Scenario: You need an array initialized with all zeros for some calculations.

Question: How do you create a NumPy array filled with zeros?

Answer:
```
arr = np.zeros((3, 4))
```

Explanation:
→ zeros(shape) creates an array of given shape filled with 0.
→ Very useful for initialization.

Visual:

Shape (3, 4)

0	0	0	0
0	0	0	0
0	0	0	0

★ Common in ML, Image Processing and DP problems (initialization).

7 Create an Array of Ones

Scenario: You need an array initialized with all ones for some operations.

Question: How do you create a NumPy array filled with ones?

Answer:
```
arr = np.ones((2, 5))
```

Explanation:
→ ones(shape) creates an array of given shape filled with 1.
→ Useful in mathematical operations and masking.

Visual:

Shape (2, 5)

1	1	1	1	1
1	1	1	1	1

★ Useful for initialization, counting and masking.

8 Generate Array with Random Numbers

Scenario: You are working on a simulation and need random numbers.

Question: How do you generate an array of random numbers?

Answer:
```
arr = np.random.rand(2, 3)
```

Explanation:
→ rand(rows, cols) generates random numbers between 0 (inclusive) and 1 (exclusive).
→ Used in simulations, ML, random sampling, etc.

Visual:

Shape (2, 3)

0.23	0.86	0.31
0.71	0.42	0.09

★ Values are in range [0, 1).

9 Generate Random Integers

Scenario: You want random integers for selecting users or samples.

Question: How do you generate an array of random integers?

Answer:
```
arr = np.random.randint(1, 100, size=(2, 4))
```

Explanation:
→ randint(low, high, size) generates integers in the range [low, high).
→ Very useful for indexing, sampling, IDs, etc.

Visual:

Shape (2, 4)

23	87	45	12
67	33	21	98

★ Range is [1, 100) (1 included, 100 excluded).

10 Create Identity Matrix

Scenario: You need an identity matrix for linear algebra or transformations.

Question: How do you create an identity matrix?

Answer:
```
arr = np.eye(4)
```

Explanation:
→ eye(n) creates an n x n identity matrix.
→ Diagonal elements are 1, others are 0.

Visual:

4 x 4 Identity Matrix

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

★ Used in linear algebra, solving equations, and ML algorithms.

11 Indexing in NumPy Array

Scenario: You have sales data and want to access a specific element.

Question: How do you access an element in a NumPy array?

Answer: `arr[1, 2]`

Explanation:
- → Indexing is used to access an element using its row and column index.
- → Indexing starts from 0.

Why it's important: It is the foundation for data extraction and manipulation.

Visual:

Array (2D)

10	20	30	40
50	60	70	80
90	100	110	120

arr[1, 2] ↓ 70

★ Row index comes first, then column index.

12 Slicing in NumPy Array

Scenario: You want to get a subset of data (e.g., first 2 rows and 3 columns).

Question: How do you slice a NumPy array?

Answer: `arr[0:2, 0:3]`

Explanation:
- → Slicing extracts a part of the array.
- → Syntax: arr[start_row:end_row, start_col:end_col]
- → End index is exclusive.

Why it's important: Helps in selecting specific data for analysis and processing.

Visual:

Original Array (3x4)

10	20	30	40
50	60	70	80
90	100	110	120

arr[0:2, 0:3] →

Result (2x3)

10	20	30
50	60	70

★ Very useful for data extraction and subsetting.

13 Boolean Indexing

Scenario: You want to filter values greater than 50 from an array.

Question: How do you use boolean indexing?

Answer: `arr[arr > 50]`

Explanation:
- → Returns all elements where condition is True.
- → Result is a 1D array.

Why it's important: Helps in filtering data based on conditions, just like WHERE clause in SQL.

Visual:

Array

10	60	30	80
40	70	20	90

Condition : arr > 50

False	True	False	True
False	True	False	True

Result

60
80
70
90

★ Extremely powerful for data filtering.

14 Filtering with Conditions

Scenario: You want to get all sales values between 50 and 100.

Question: How do you filter values in a range?

Answer: `arr[(arr >= 50) & (arr <= 100)]`

Explanation:
- → Combine conditions using & (and), | (or).
- → Use parentheses to avoid ambiguity.

Why it's important: Frequently used in real-world data analysis and dashboards.

Visual:

Array

20	55	70	120	85
45	60	95	30	100

Condition : (arr >= 50) & (arr <= 100)

False	True	True	False	True
False	True	True	False	True

Result

55
70
85
60
95
100

★ Helps in extracting data within a range.

15 Reshape Array

Scenario: You have a 1D array and want to convert it into a 2D array.

Question: How do you reshape an array?

Answer: `arr.reshape(2, 3)`

Explanation:
- → Reshape changes the shape without changing the data.
- → Total elements must remain the same.

Why it's important: Useful in data transformation, ML models and preparing data for algorithms.

Visual:

Original 1D Array (6,)

1	2	3	4	5	6

arr.reshape(2, 3) →

Reshaped 2D Array (2x3)

1	2	3
4	5	6

★ Essential in ML, image processing and matrix ops.

16 Flatten Array

Scenario: You have a multi-dimensional array and want to convert it into 1D.

Question: How do you flatten a NumPy array?

Answer: `arr.flatten()`

Explanation:
→ Converts any n-dimensional array into a 1D array (copy).
→ Also use arr.ravel() (returns view if possible).

Why it's important: Useful when data needs to be fed into algorithms or models that require 1D input.

Visual:

Original Array (2x3)

1	2	3
4	5	6

↓

Flattened (1D)

1	2	3	4	5	6

★ Converts multi-dimensional array into 1D array.

17 Transpose Array

Scenario: You need to interchange rows and columns of a 2D array.

Question: How do you transpose a NumPy array?

Answer: `arr.T`

Explanation:
→ Transposes rows to columns and columns to rows.
→ For 2D arrays, arr.T is commonly used.

Why it's important: Important in linear algebra, matrix operations and data preprocessing.

Visual:

Original Array (2x3)

1	2	3
4	5	6

→

Transposed (3x2)

1	4
2	5
3	6

★ Rows become columns and columns become rows.

18 Stack Arrays Vertically

Scenario: You have two arrays and want to combine them row-wise.

Question: How do you stack arrays vertically?

Answer: `np.vstack((arr1, arr2))`

Explanation:
→ Stacks arrays one on top of another (row-wise).
→ Number of columns must be the same.

Why it's important: Used to combine datasets, add new records, append rows.

Visual:

arr1 (2x3)

1	2	3
4	5	6

arr2 (1x3)

7	8	9

→

Result (3x3)

1	2	3
4	5	6
7	8	9

★ Stacks arrays vertically (row-wise).

19 Stack Arrays Horizontally

Scenario: You have two arrays and want to combine them column-wise.

Question: How do you stack arrays horizontally?

Answer: `np.hstack((arr1, arr2))`

Explanation:
→ Stacks arrays side by side (column-wise).
→ Number of rows must be the same.

Why it's important: Useful to add new features/columns, merge data side by side.

Visual:

arr1 (2x2)

1	2
3	4

arr2 (2x2)

5	6
7	8

→

Result (2x4)

1	2	5	6
3	4	7	8

★ Stacks arrays horizontally (column-wise).

20 Split Array

Scenario: You have a large array and want to split it into smaller parts.

Question: How do you split a NumPy array?

Answer: `np.array_split(arr, 3)`

Explanation:
→ Splits array into 3 equal (or nearly equal) parts along given axis.
→ Another option: np.split(arr, indices)

Why it's important: Useful for parallel processing, cross-validation, and managing large data.

Visual:

Original Array (1D, 9 elements)

1	2	3	4	5	6	7	8	9

↓ np.array_split(arr, 3)

Part 1

1	2	3

Part 2

4	5	6

Part 3

7	8	9

★ Splits array into multiple sub-arrays.

Reshur AI

21 Concatenate Arrays

Scenario: You have two arrays and want to join them along an existing axis.

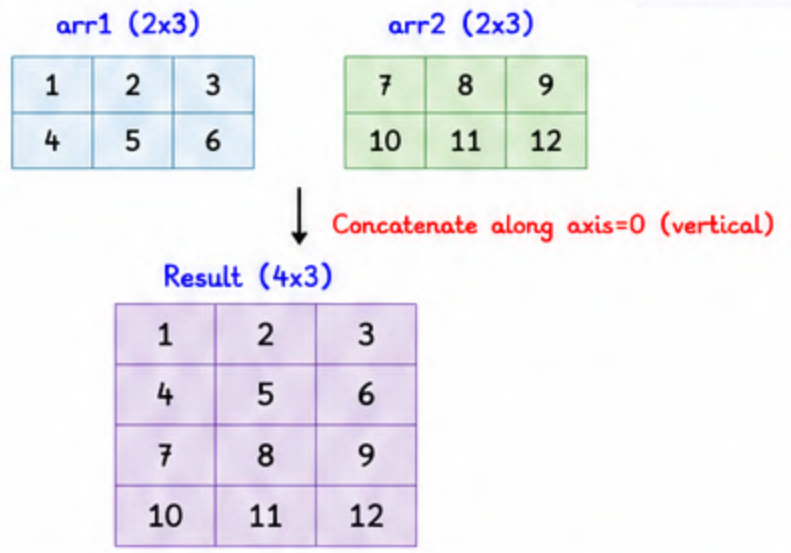
Question: How do you concatenate two NumPy arrays?

Answer:
```
np.concatenate((arr1, arr2), axis=0)
```

Explanation:
- → Joins arrays along the specified axis.
- → axis=0 → vertical (row-wise), axis=1 → horizontal (column-wise).

Why it's important: Useful when combining datasets of same columns or rows.

Visual:



22 Delete Elements from Array

Scenario: You want to remove some unwanted elements from a NumPy array.

Question: How do you delete elements?

Answer:
```
np.delete(arr, obj, axis=None)
```

Explanation:
- → Removes specified index or indices (obj) from array.
- → axis=None → array is flattened before deletion.

Why it's important: Helps in data cleaning and preprocessing.

Visual:



23 Find Maximum and Minimum

Scenario: You have numerical data and want to find the highest and lowest values.

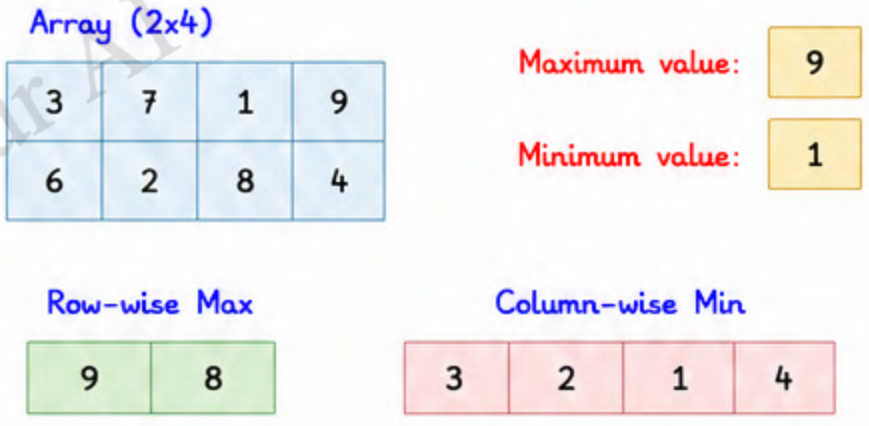
Question: How do you find max and min values?

Answer:
```
arr.max()  # maximum value
arr.min()  # minimum value
```

Explanation:
- → Returns the maximum or minimum element.
- → You can also use axis to get row-wise or column-wise results.

Why it's important: Essential for analysis, reporting and decision making.

Visual:



Resnur AI

24 Compute Mean, Median and Standard Deviation

Scenario: You want to understand the summary statistics of your data.

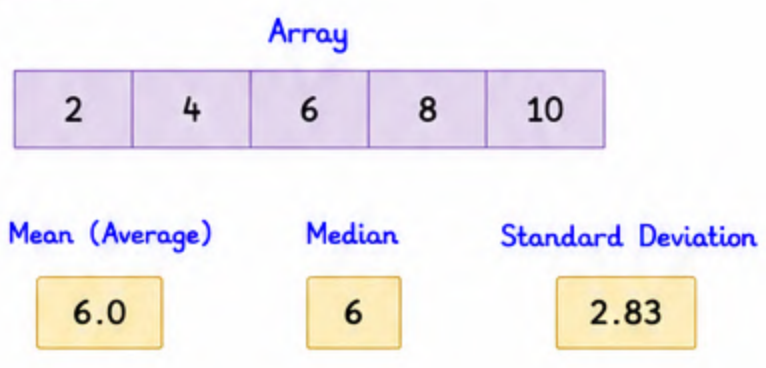
Question: How do you compute mean, median and standard deviation?

Answer:
```
arr.mean()      # mean
np.median(arr)  # median
arr.std()       # standard deviation
```

Explanation:
- → mean() → average of values.
- → median() → middle value.
- → std() → spread or variation in data.

Why it's important: Widely used in data analysis, ML and statistical reporting.

Visual:



25 Clip Values in Array

Scenario: You have data with outliers and want to limit the values within a range.

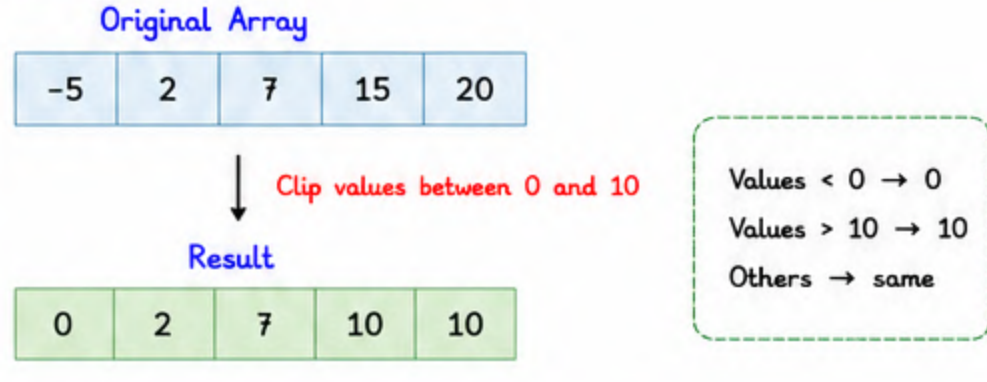
Question: How do you clip values in a NumPy array?

Answer:
```
np.clip(arr, a_min, a_max)
```

Explanation:
- → Values less than a_min become a_min.
- → Values greater than a_max become a_max.
- → Other values remain unchanged.

Why it's important: Helps in handling outliers and keeping data within defined limits.

Visual:



26 Unique Elements in Array

Scenario: You have an array with duplicate values and want to find only the unique values.

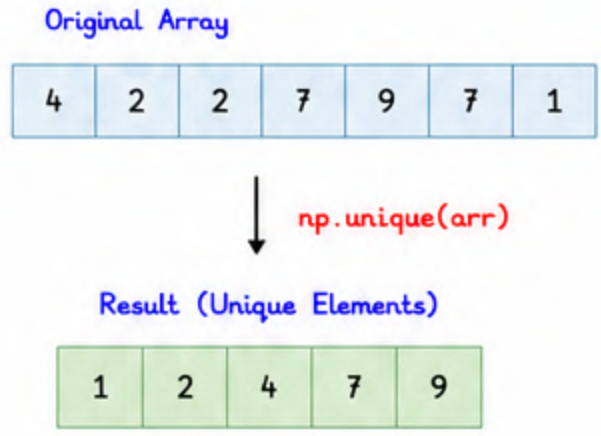
Question: How do you get unique elements in a NumPy array?

Answer:
```
np.unique(arr)
```

Explanation:
→ Returns the sorted unique elements of the array.
→ Useful for data summarization and analysis.

Why it's important: Helps in removing duplicates and understanding distinct values in data.

Visual:



27 Sort an Array

Scenario: You want to sort the elements of an array in ascending or descending order.

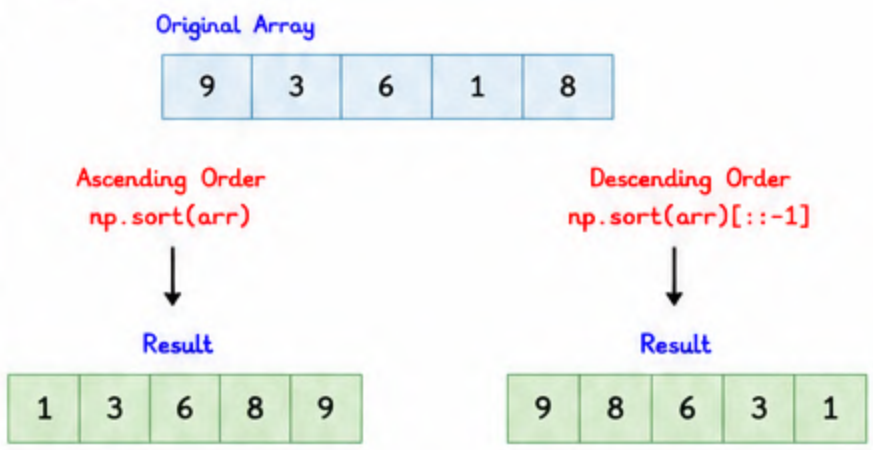
Question: How do you sort a NumPy array?

Answer:
```
np.sort(arr)
np.sort(arr)[::-1]  # descending order
```

Explanation:
→ np.sort(arr) returns a sorted copy (ascending).
→ [::-1] reverses the array for descending order.

Why it's important: Sorting is widely used in data analysis, searching and ranking.

Visual:



28 Sum of Array Elements

Scenario: You have numerical data and want to find the total sum of all elements.

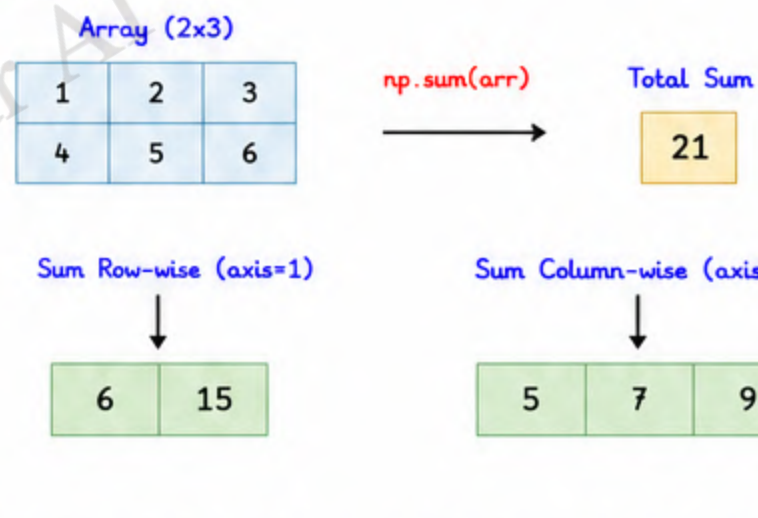
Question: How do you compute the sum of elements in a NumPy array?

Answer:
```
np.sum(arr)
```

Explanation:
→ Returns the sum of all elements in the array.
→ You can also use axis to sum row-wise or column-wise.

Why it's important: Used in aggregations, reporting, and mathematical computations.

Visual:



29 Dot Product of Two Arrays

Scenario: You have two vectors and want to calculate their dot (inner) product.

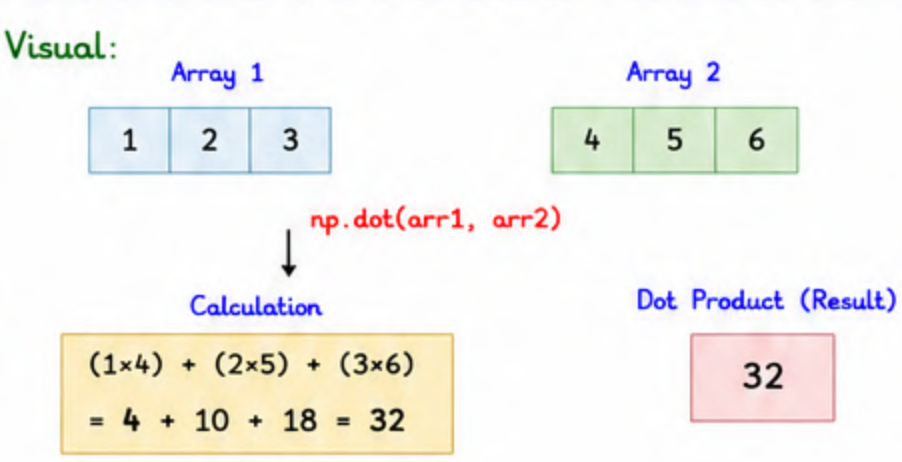
Question: How do you compute the dot product using NumPy?

Answer:
```
np.dot(arr1, arr2)
```

Explanation:
→ Multiplies corresponding elements and sums them up.
→ Both arrays must have the same length.

Why it's important: Essential in linear algebra, machine learning and vector computations.

Visual:



30 Broadcasting in NumPy

Scenario: You want to perform operations on arrays of different shapes.

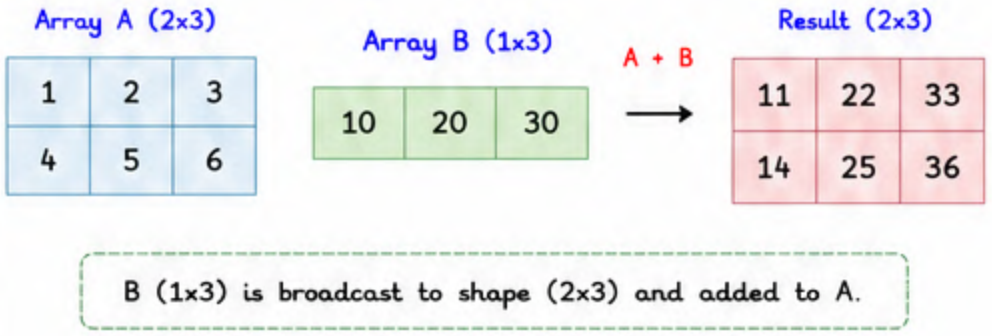
Question: How does broadcasting work in NumPy?

Answer: NumPy automatically expands the smaller array to match the shape of the larger one.

Explanation:
→ Allows arithmetic operations on arrays with compatible shapes.
→ Saves time and memory (no need to copy data).

Why it's important: Makes code efficient, concise and powerful in numerical computations.

Visual:



ResnurAI

31 Element-wise Operations

Scenario: You have two arrays and want to perform element-wise operations on them.

Question: How do you add, subtract, multiply and divide two NumPy arrays element-wise?

Answer:

```
arr1 + arr2    # addition
arr1 - arr2    # subtraction
arr1 * arr2    # multiplication
arr1 / arr2    # division
```

Explanation:
- → Operations are performed element by element.
- → Both arrays must have the same shape.

Why it's important: Very useful in mathematical computations, data transformations and ML models.

Visual:

Array 1

1	2	3

Array 2

4	5	6

Addition (arr1 + arr2)	→	5	7	9
Subtraction (arr1 - arr2)	→	-3	-3	-3
Multiplication (arr1 * arr2)	→	4	10	18
Division (arr1 / arr2)	→	0.25	0.4	0.5

32 Matrix Multiplication

Scenario: You are working with matrices and want to perform matrix (dot) multiplication.

Question: How do you multiply two 2D arrays (matrices) in NumPy?

Answer:

```
np.dot(arr1, arr2)
or
arr1 @ arr2
```

Explanation:
- → Performs matrix multiplication (dot product).
- → Number of columns in arr1 must be equal to number of rows in arr2.

Why it's important: Essential in ML, linear algebra, deep learning and data science.

Visual:

Matrix A (2x3)

1	2	3
4	5	6

×

Matrix B (3x2)

7	8
9	10
11	12

→

Result (2x2)

58	64
139	154

Calculation:

```
58 = (1×7) + (2×9) + (3×11)
64 = (1×8) + (2×10) + (3×12)
139 = (4×7) + (5×9) + (6×11)
154 = (4×8) + (5×10) + (6×12)
```

33 Find Index of Maximum and Minimum

Scenario: You have an array and want to find the index positions of the largest and smallest values.

Question: How do you get the index of max and min elements?

Answer:

```
np.argmax(arr)    # index of max
np.argmin(arr)    # index of min
```

Explanation:
- → Returns the index position of the first occurrence.
- → For 2D arrays, use axis to specify direction.

Why it's important: Helps in data analysis, optimization and decision making.

Visual:

Array

10	3	25	7	18

np.argmax(arr) → 2 (value = 25)

np.argmin(arr) → 1 (value = 3)

For 2D Array

```
arr = [[1, 8, 2],
       [6, 3, 9]]
```

np.argmax(arr) → 5 (value = 9 at index 5)

np.argmin(arr) → 0 (value = 1 at index 0)

(Flattened index)

34 Repeat Elements in Array

Scenario: You want to repeat elements of an array multiple times.

Question: How do you repeat each element in a NumPy array?

Answer:

```
np.repeat(arr, repeats)
or
np.repeat(arr, repeats, axis=0/1)
```

Explanation:
- → By default, repeats each element.
- → Use axis to repeat along rows or columns for 2D arrays.

Why it's important: Useful in data expansion, balancing datasets and signal processing.

Visual:

1D Array

1	2	3

np.repeat(arr, 3) →

Result

1	1	1	2	2	2	3	3

2D Array

1	2
3	4

np.repeat(arr, 2, axis=0) →

Repeat Rows

1	2
1	2
3	4
3	4

np.repeat(arr, 2, axis=1) →

Repeat Columns

1	1	2	2
3	3	4	4

35 Round Off Values

Scenario: You have floating point values and want to round them to a specific number of decimals.

Question: How do you round off values in a NumPy array?

Answer:

```
np.round(arr, decimals)
```

Explanation:
- → Rounds the values to the specified number of decimal places.

Why it's important: Useful in reporting, presentation, reducing noise and financial calculations.

Visual:

Original Array

3.14159	2.71828	1.23456	9.87654

np.round(arr, 2) → Rounded to 2 decimals

3.14	2.72	1.23	9.88

np.round(arr, 0) → Rounded to 0 decimals

3.0	3.0	1.0	10.0

Resnur AI

36 Reshape Array

Scenario: You have an array and want to change its shape without changing its data.

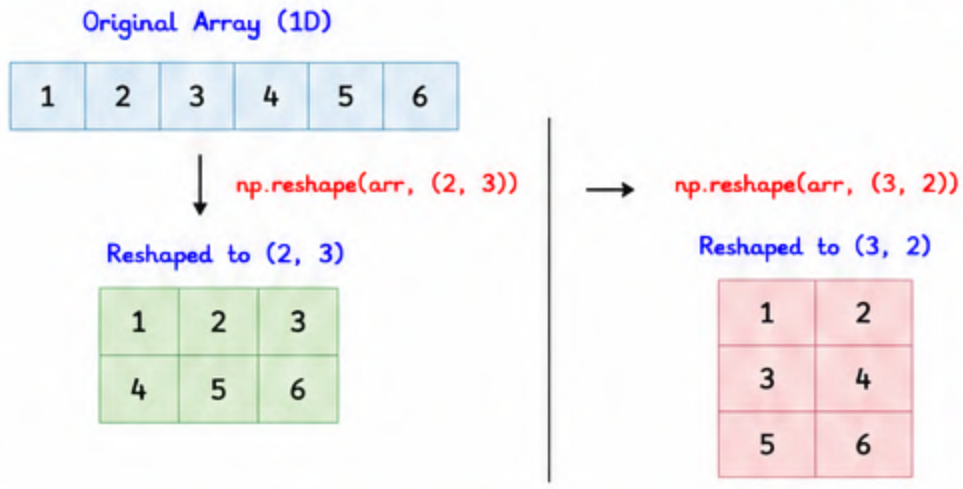
Question: How do you reshape a NumPy array?

Answer:
```
np.reshape(arr, new_shape)
or
arr.reshape(new_shape)
```

Explanation:
- → Changes the shape of the array.
- → Total number of elements must remain the same.

Why it's important: Helps in data transformation, preparing data for ML models and matrix operations.

Visual:



37 Flatten Multi-dimensional Array

Scenario: You have a multi-dimensional array and want to convert it into a 1D array.

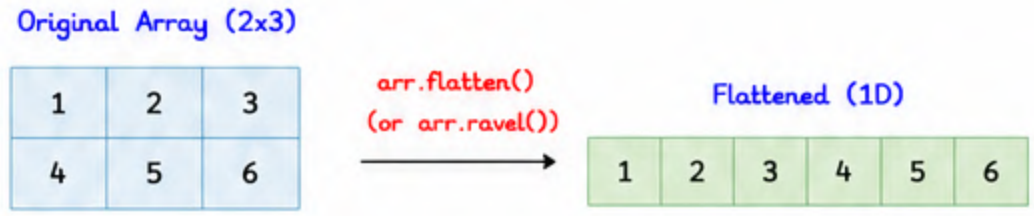
Question: How do you flatten a multi-dimensional NumPy array?

Answer:
```
arr.flatten()
or
arr.ravel()
```

Explanation:
- → Converts any n-dimensional array into 1D.
- → arr.ravel() returns a view (if possible), arr.flatten() returns a copy.

Why it's important: Useful for feeding data to algorithms that require 1D input.

Visual:



38 Extract Diagonal Elements

Scenario: You have a 2D square matrix and want to extract its diagonal elements.

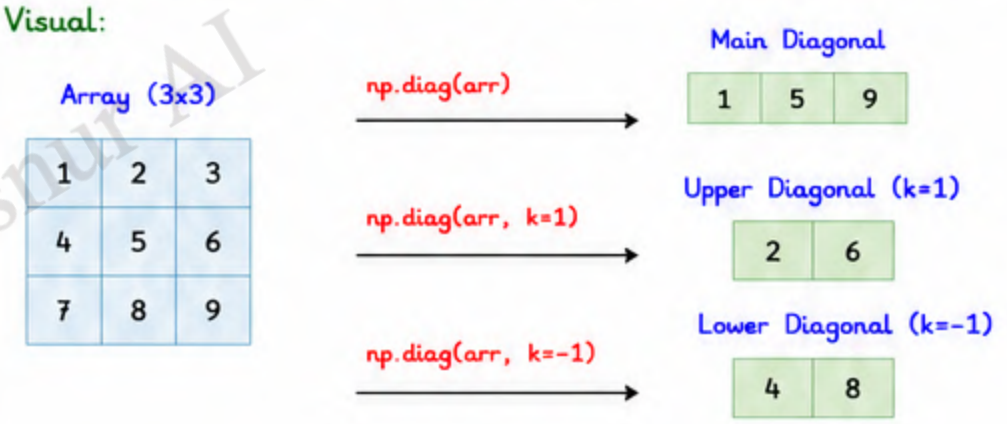
Question: How do you get the diagonal elements from a NumPy array?

Answer: `np.diag(arr)`

Explanation:
- → Returns the main diagonal (top-left to bottom-right) elements of a 2D array.
- → Use np.diag(arr, k) for other diagonals.

Why it's important: Important in linear algebra, matrix analysis and solving equations.

Visual:



Resnur AI

39 Reverse Array

Scenario: You want to reverse the elements of an array.

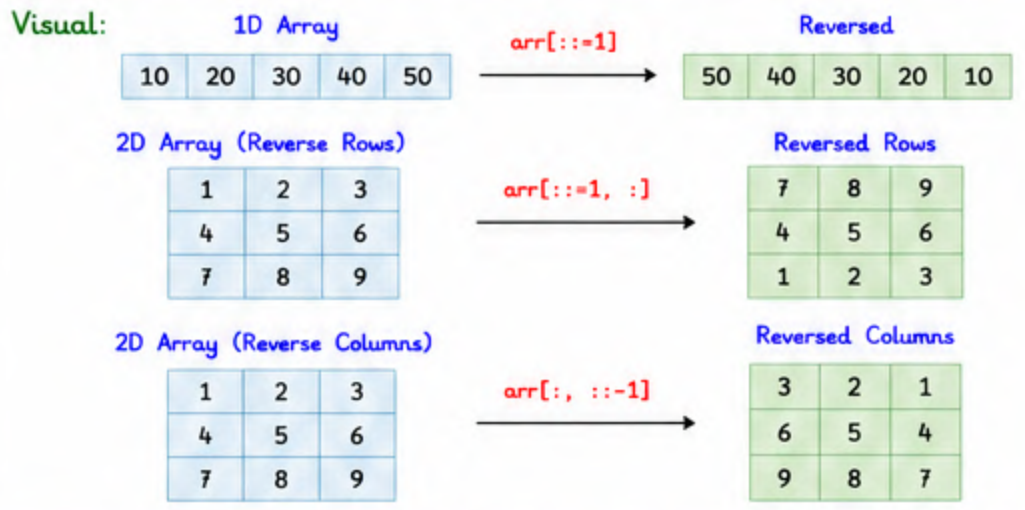
Question: How do you reverse a NumPy array?

Answer: `arr[::-1]`

Explanation:
- → Reverses the array along the first axis.
- → For 2D arrays, use arr[::-1, :] to reverse rows or arr[:, ::-1] to reverse columns.

Why it's important: Useful in data preprocessing, time series analysis and algorithmic problems.

Visual:



40 Create Array from Existing Data

Scenario: You have Python lists, tuples or other arrays and want to create a NumPy array from it.

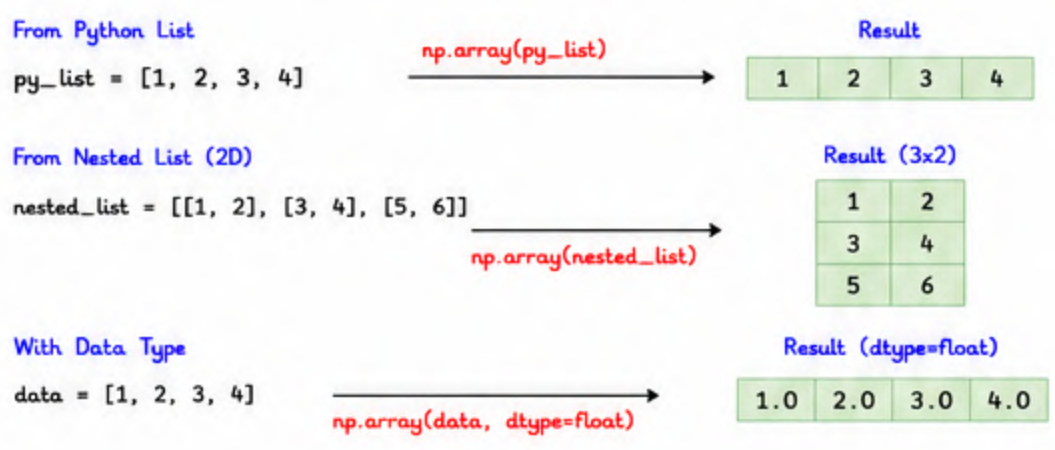
Question: How do you create a NumPy array from existing data?

Answer: `np.array(data, dtype=...)`

Explanation:
- → Converts lists, tuples or other array-like objects into a NumPy array.
- → dtype (optional) specifies the data type.

Why it's important: Essential first step in working with data using NumPy.

Visual:



41 Identity Matrix

Scenario: You want to create an identity matrix of size n × n.

Question: How do you create an identity matrix using NumPy?

Answer:

```
np.eye(n)
np.identity(n)
```

Explanation:
→ Creates an n × n identity matrix.
→ Diagonal elements are 1 and others are 0.

Why it's important: Used in linear algebra, transformations and solving equations.

Visual:

n = 4

np.eye(4)

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

42 Generate Random Numbers

Scenario: You need random numbers for simulation or sampling.

Question: How do you generate random numbers in NumPy?

Answer:

```
np.random.rand()      # uniform [0,1)
np.random.randn()     # standard normal
np.random.randint(low, high, size)
```

Explanation:
→ rand() generates uniform random numbers [0, 1).
→ randn() generates standard normal distribution.
→ randint() generates random integers.

Why it's important: Essential in simulations, ML, sampling, and testing.

Visual:

Uniform [0, 1): np.random.rand(2, 3)

0.23	0.87	0.11
0.65	0.42	0.98

Standard Normal: np.random.randn(2, 3)

-0.27	1.12	0.36
0.82	-1.45	-0.33

Random Integers: np.random.randint(1, 10, size=(2, 4))

7	2	9	4
1	8	3	6

43 Cumulative Sum and Cumulative Product

Scenario: You want to compute running totals and running products.

Question: How do you calculate cumulative sum and cumulative product in NumPy?

Answer:

```
np.cumsum(arr)
np.cumprod(arr)
```

Explanation:
→ cumsum() returns cumulative sum.
→ cumprod() returns cumulative product.

Why it's important: Useful in time series, finance, signal processing and analytics.

Visual:

Array

1	2	3	4	5

↓

Cumulative Sum: np.cumsum(arr)

1	3	6	10	15

↓

Cumulative Product: np.cumprod(arr)

1	2	6	24	120

Reshur AI

44 Save Array to File and Load Array

Scenario: You want to save a NumPy array to a file and load it later.

Question: How do you save and load arrays in NumPy?

Answer:

```
np.save('file.npy', arr)
arr = np.load('file.npy')
```

Explanation:
→ save() saves array in .npy binary format.
→ load() loads the array from the file.

Why it's important: Important for data persistence, sharing and reusability.

Visual:

Original Array

1	2	3	4	5

Save to file:
np.save('data.npy', arr) → data.npy

Load from file:
arr = np.load('data.npy') →

Loaded Array

1	2	3	4	5

45 Find All Indices of a Value

Scenario: You want to find all positions where a specific value occurs in an array.

Question: How do you get all indices of a value?

Answer:

```
np.where(arr == value)
np.argwhere(arr == value)
```

Explanation:
→ where() returns tuple of arrays of indices.
→ argwhere() returns indices in (row, col) format.

Why it's important: Very useful in searching, filtering, masking and data analysis.

Visual:

1D Array

4	7	2	7	3	7

Value = 7

np.where(arr == 7)

```
(array([1, 3, 5]),)
```

np.argwhere(arr == 7)

```
[[1]
 [3]
 [5]]
```

45 Find All Indices of a Value

Scenario: You want to find all positions where a specific value occurs in an array.

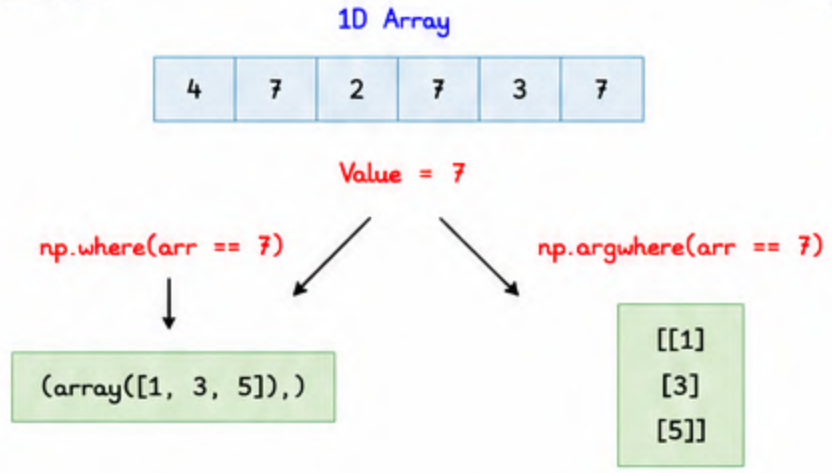
Question: How do you get all indices of a value?

Answer:
```
np.where(arr == value)
np.argwhere(arr == value)
```

Explanation:
→ where() returns tuple of arrays of indices.
→ argwhere() returns indices in (row, col) format.

Why it's important: Very useful in searching, filtering, masking and data analysis.

Visual:



46 Count Occurrences of Each Value

Scenario: You want to count how many times each unique value appears.

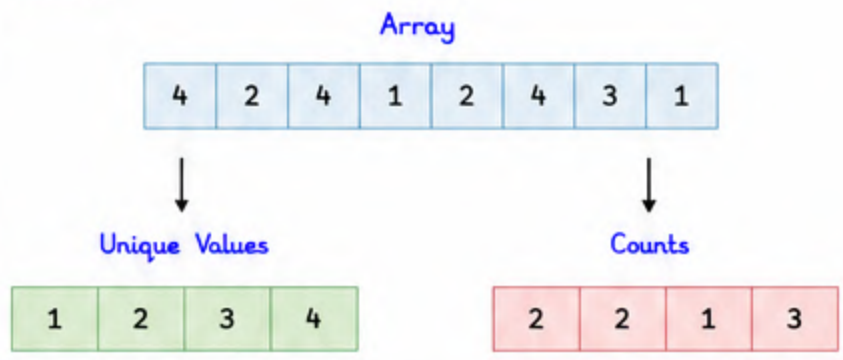
Question: How do you count occurrences of values in a NumPy array?

Answer:
```
unique_vals, counts = np.unique(arr, return_counts=True)
```

Explanation:
→ unique() finds unique elements.
→ return_counts=True returns counts of each.

Why it's important: Helps in frequency analysis, categorization and data summarization.

Visual:



47 Check if Array Contains a Value

Scenario: You want to check whether a particular value exists in the array.

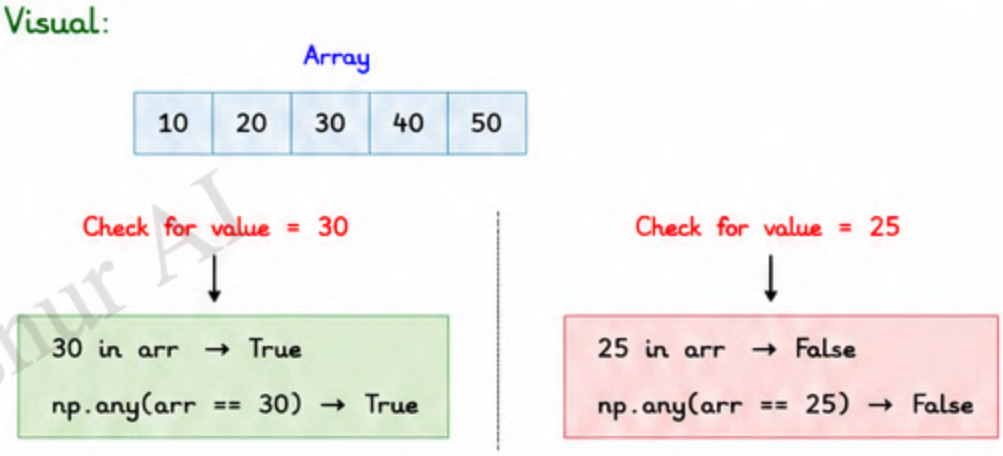
Question: How do you check if an array contains a value?

Answer:
```
value in arr
np.any(arr == value)
```

Explanation:
→ 'in' checks membership (for 1D arrays).
→ any(arr == value) works for any shape.

Why it's important: Quick check for validation and filtering.

Visual:



Resmur AI

48 Convert Data Type of Array

Scenario: You want to change the data type of array elements.

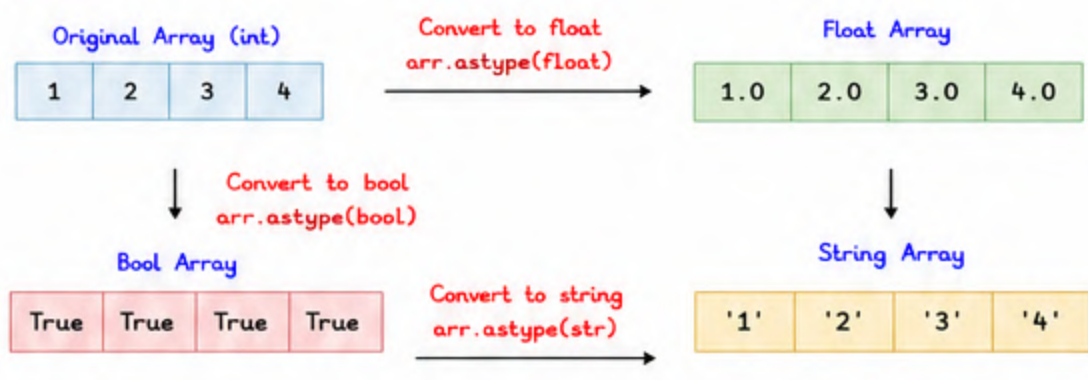
Question: How do you convert the data type of a NumPy array?

Answer:
```
arr.astype(new_dtype)
```

Explanation:
→ astype() returns a copy of array with specified data type.
→ Common types: int, float, bool, str, etc.

Why it's important: Useful for memory optimization and ensuring correct computations.

Visual:



49 Filter Array with Condition

Scenario: You want to filter elements that satisfy a specific condition.

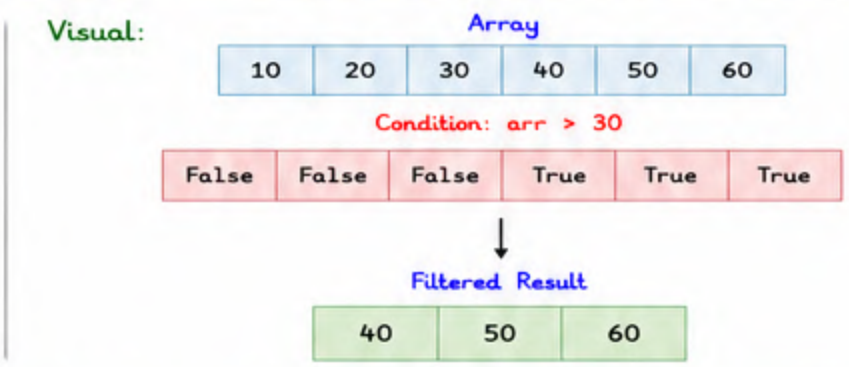
Question: How do you filter an array using a condition?

Answer:
```
filtered_arr = arr[condition]
```

Explanation:
→ condition is a boolean array (True/False).
→ Returns elements where condition is True.

Why it's important: Essential for data filtering, feature selection and analysis.

Visual:



50 Replace Elements in Array

Scenario: You want to replace certain values in an array with new values.

Question: How do you replace elements in a NumPy array?

Answer:
```
np.where(condition, new_value, arr)
```

Explanation:
→ If condition is True → use new_value.
→ If condition is False → keep original value.

Why it's important: Useful in data cleaning, transformation and rule-based updates.

Visual:

